Software-Handbuch

Dokumentation für das Softwareprojekt *Monaa-Lisa*

Version 0.1

19. November 2025

Autor: Nico Bestek

Team: Bastian Rosinski, Lenio Cabral Rosario, Nick Wittkowski

Inhaltsverzeichnis

1 Einführung **1**
1.1 Zweck dieses Handbuchs 1
1.2 Überblick über die Software 1

2 Installation und Systemvoraussetzungen **2**
2.1 Systemvoraussetzungen 2
2.2 Installationsschritte 2

3 Erste Schritte **3**
3.1 Programm starten und aufrufen 3

4 Benutzeroberfläche **4**
4.1 Hauptfenster 4

5 Funktionen im Detail **5**
5.1 Funktion A 5
5.2 Funktion B 5
5.3 Beispiel: API-Aufruf 5

6 Fehlersuche (Troubleshooting) **6**
6.1 Häufige Probleme und Lösungen 6

7 Anhang **7**
7.1 Konfigurationsdateien 7

1 Einführung

1.1 Zweck dieses Handbuchs

Dieses Handbuch beschreibt Installation, Einrichtung und Nutzung des im Rahmen des Softwarepraktikums 2025 entwickelten Tools *Monaa-Lisa*. Es soll allen Beteiligten eine verständliche und zusammenhängende Hilfe biieten.

Adressiert werden insbesondere:

- Anwenderinnen und Anwender, die Monaa-Lisa im Alltag einsetzen,
- Administratorinnen und Administratoren, die das System bereitstellen und betreiben,
- Entwicklerinnen und Entwickler, die die Software erweitern oder anpassen möchten.

Ziel ist es, die notwendigen Schritte von den Systemvoraussetzungen über die Erstinstallation bis hin zur Fehleranalyse nachvollziehbar zu dokumentieren und häufige Fragen sowie typische Probleme an einer zentralen Stelle zu bündeln.

1.2 Überblick über die Software

Monaa-Lisa ist als Fullstack-Applikation entworfen und besteht aus drei Teilen: Frontend, Backend und Datenbank. Die Komponenten kommunizieren über Schnittstellen miteinander und folgen dem Prinzip einer *Single Source of Truth*: Alle Daten werden in der Datenbank gespeichert und von dort aus den übrigen Teilsystemen bereitgestellt.

Datenbank. Die Datenhaltung erfolgt in einer relationalen Datenbank auf Basis des Datenbankmanagementsystems PostgreSQL. Sie speichert sämtliche Daten und stellt diese auf Anfrage den anderen Systemen bereit. Die Datenbank ist bewusst möglichst „dumm“ gestaltet: Ihre Aufgabe beschränkt sich auf Speicherung und Bereitstellung der Daten. Logik und Datenmanipulationen werden im Backend umgesetzt.

Backend. Das Backend ist in TypeScript unter Verwendung des Frameworks NestJS implementiert. Es kommuniziert direkt mit der Datenbank und kapselt die Logik der Anwendung. Über HTTP- bzw. API-Endpunkte stellt das Backend Schnittstellen für das Frontend bereit. Eingehende Anfragen werden validiert und je nach Berechtigung und Inhalt an die Datenbank weirgereicht oder verworfen.

Frontend. Das Frontend basiert auf Svelte in Kombination mit Vite als Build-Tool. Für die grafische Darstellung von Graphen und Beziehungen wird die Bibliothek `sigma` eingesetzt. Das Frontend kommuniziert über HTTP-Requests mit dem Backend und erhält die Daten in der Regel im JSON-Format, um sie anschließend benutzerfreundlich aufzubereiten und interaktiv darzustellen.

2 Installation und Systemvoraussetzungen

2.1 Systemvoraussetzungen

Monaa-Lisa ist für den Betrieb in einer containerisierten Umgebung konzipiert und kann grundsätzlich auf jedem System ausgeführt werden, das Docker unterstützt. Aufgrund der komplexen Verwendung von künstlicher Intelligenz und grafische Aufbereitung der Daten muss das zu verwendende Gerät über eine gewisse Menge an Resourcen verfügen, die einen normalen Öfifce-PC"übersteigen.

- **Unterstützte und getestete Betriebssysteme:** Ubuntu 24.04 LTS, Windows 10, Windows 11.
- **Empfohlene Hardware:** CPU (mindestens 4 Kerne), mindestens 16 GB RAM, GPU mit mindestens 4 GB VRAM für rechenintensive Aufgaben.
- **Erforderliche Drittsoftware:** Docker (inklusive Docker Compose), Node.js, Python 3.x sowie ein aktueller Webbrowser.

2.2 Installationsschritte

Die Installation von Monaa-Lisa erfolgt in mehreren aufeinander aufbauenden Schritten:

1. **GitHub-Repository klonen**
   Klonen Sie das offizielle Monaa-Lisa-Repository.
2. **Python-Abhängigkeiten installieren**
   Installieren Sie alle benötigten Python-Pakete über `requirements.txt`
3. **NPM-Abhängigkeiten installieren**
   Installieren Sie anschließend die JavaScript-Abhängigkeiten für Frontend und Backend über `npm install`
4. **Umgebungsvariablen setzen**
   Legen Sie die erforderlichen Umgebungsvariablen (z. B. Datenbankzugang, Ports) in einer `.env`-Datei an oder konfigurieren Sie sie direkt im System. Achten Sie darauf, dass die Werte mit der Docker- und Datenbankkonfiguration übereinstimmen. Unten ist eine Beispiel .env Datei.
5. **Docker-Umgebung starten**
   Stellen Sie sicher, dass Docker und Docker Compose installiert sind, und starten Sie die Container über `docker compose up`. Dadurch werden insbesondere Datenbank und ggf. weitere Backend-Dienste bereitgestellt.
6. **Backend starten**
   Starten Sie den Backend-Server
7. **Frontend bauen und starten**
   Bauen Sie das Frontend und starten Sie es über das vorgesehene Build- bzw. Startkommando (z. B. `npm run build` oder `npm run dev`). Anschließend kann die Anwendung über einen Webbrowser aufgerufen werden.

3 Erste Schritte

3.1 Programm starten und aufrufen

Rücksprache mit den anderen

4 Benutzeroberfläche

4.1 Hauptfenster

Rücksprache mit anderen Mitgliedern

5 Funktionen im Detail

5.1 Funktion A

Rücksprache mit anderen Mitgliedern

5.2 Funktion B

Rücksprache mit anderen Mitgliedern

5.3 Beispiel: API-Aufruf

```python
def get_all_embeddings():
    session = SessionLocal()
    try:
        limit = cfg.get_int(
            "semanticpaper",
            "embeddings_preload_limit",
            int(os.getenv("EMBEDDINGS_PRELOAD_LIMIT", "5000"))
        )
        q = session.query(DBEmbedding)
        if limit > 0:
            q = q.limit(limit)
        try:
            db_embeddings_rows = q.all()
        except ProgrammingError as e:
            logger.warning(f"Embeddings table missing or not initialized yet: {e}")
            return {}
        # Get rid of the warning in pycharm
        db_embeddings = cast(List[DBEmbedding], db_embeddings_rows)
        embedding_dict = {
            db_embedding.belonging_paper_entry_id: Embedding.from_db(db_embedding)
            for db_embedding in db_embeddings if db_embedding.content is not None
        }
        return embedding_dict
    finally:
        session.close()
```

Listing 5.1: Beispiel: API-Aufruf

6 Fehlersuche (Troubleshooting)

6.1 Häufige Probleme und Lösungen

Problem: Keine Verbindung zum Server
Unbedingt Ports prüfen. Ggf. Firewall Einstellungen prüfen

Problem: Datenbank kann nicht erreicht werden
Zugangsdaten und Passwort in der .env Datei prüfen.

7 Anhang

7.1 Konfigurationsdateien

```
POSTGRES_USER=monaa
POSTGRES_PASSWORD=sicherespasswort
POSTGRES_DB=monaa_lisa_db
```

Listing 7.1: Beispiel: .env